

Лабораторная работа №1. Вычисление числа π с использованием OpenMP

1. Описание работы

На языке C реализованы последовательная и параллельная версия программы вычисления числа π с использованием ряда Лейбница. Параллельная версия использует директиву `#pragma omp parallel for reduction(+:sum)`, время выполнения измеряется функцией `omp_get_wtime()`, ускорение вычисляется с помощью формулы $S = T_{\text{посл}} / T_{\text{пар.}}$. Эксперименты проводились на процессоре Intel Core i5-12450H (8 ядер: 4 производительных P-ядра и 4 энергоэффективных E-ядра, 12 логических потоков) в WSL Ubuntu с компилятором GCC.

2. Результаты экспериментов

Таблица 1 – Влияние количества потоков (при $n = 10^9$)

Потоки	Время, с	Ускорение
1	1.8734	1.00
2	0.9745	1.93
4	0.6042	3.33
8	0.4680	4.41
16	0.3742	5.12

Таблица 2 – Влияние размера задачи (при 8 потоках)

n	Последовательно, с	OpenMP (8 потоков), с	Ускорение
10^7	0.0210	0.0315	0.67
10^8	0.1902	0.0670	2.84
10^9	1.8733	0.4074	4.60
10^{10}	18.7603	3.8820	4.83

3. Вывод

При увеличении числа потоков время выполнения для $n = 10^9$ уменьшалось: с 1,87 с на одном потоке до 0,37 с на 16 потоках. На 2 потоках ускорение было близко к идеальному – эффективность около 97 %, на 4 потоках – 83 %. После 4 потоков рост ускорения замедлился: 8 потоков дали лишь эффективность 55 %, а 16 потоков – 32 %. Скорее всего, это связано с тем, что в моём процессоре только 4 производительных ядра, а остальные 4 ядра энергоэффективные и работают медленнее.

Во втором эксперименте при $n = 10^7$ параллельная версия работала даже медленнее последовательной (ускорение 0,67). Это произошло потому, что задача слишком маленькая и время на создание потоков сравнимо со временем самих вычислений. При увеличении n ускорение растёт, и для $n = 10^9$ и 10^{10} оно получается около 4,6 – 4,8. Из этого можно сделать вывод, что OpenMP имеет смысл использовать для больших задач, а для маленьких выигрыша почти нет.